Feature article · Submitted to IEEE Software, Special Issue: Building Trustworthy Software in the Time of AI

Route Inside the Boundary First: Secure, Cost-Aware Routing for Enterprise LLM Inference

Pujitha Sri Lakshmi Paladugu
Independent Researcher, Lynnwood, Washington, USA
pujitha.rathna@gmail.com · ORCID 0009-0001-5678-4022

Abstract—Enterprises that adopt large language models face two coupled constraints: inference cost and data governance. Model-routing research cuts cost by sending easy requests to cheap models, but it treats where a prompt's data goes as incidental. We argue that the data boundary should be the first routing decision, not a filter added later. We present a routing architecture that classifies sensitivity before any cost optimization, keeps restricted prompts on self-hosted models by construction, escalates only non-sensitive hard requests to frontier models, and meters data egress as a first-class metric. A simple analytical model shows that savings depend at least as much on the escalation rate as on model price, that escalation also drives data egress, and that silent quality regression, not visible failure, is the risk to monitor. We close with three practices teams can adopt now.

Keywords: large language models, model routing, inference cost, data governance, zero trust, AI engineering

The two budgets every enterprise LLM team manages

Once an LLM feature leaves the pilot stage, two budgets start to bind. The first is money. Serving every request with a frontier model is simple and expensive, and at organization scale the bill quickly becomes a line item that finance asks about. The second is data. Many enterprise prompts carry customer records, source code, contracts or health information that the organization is obliged not to send to a third party, whatever that party promises about retention.

These budgets are usually owned by different people. Platform teams chase cost with routers and caches; security teams chase data with allow-lists and policy reviews. The research literature mirrors the split. Model cascades and routers such as FrugalGPT, RouteLLM, Hybrid LLM and AutoMix send easy prompts to cheaper models and escalate hard ones, and they report large savings at small quality cost [1–4]. But their objective is a cost-quality trade-off; the destination of the data is not part of the problem they solve.

In the enterprise, the destination of the data is often the dominant constraint. Treating it as a hard boundary changes the design of the router, its metrics and its failure modes. This article describes a routing architecture built around that idea, gives a back-of-the-envelope model that tells teams where their savings will actually come from, and distills three practices for teams building LLM features that must be both affordable and trustworthy.

Make the boundary the first decision

Figure 1 shows the pipeline. Every request passes through a sensitivity classifier before anything else. The classifier assigns one of three tiers that map onto most corporate data-classification policies: *restricted* (regulated or contractually confined data, including personal data, credentials and source code), which must never leave the organization's boundary; *internal*, which may leave only under an approved data-processing agreement, typically after de-identification; and *public*, which may go anywhere.

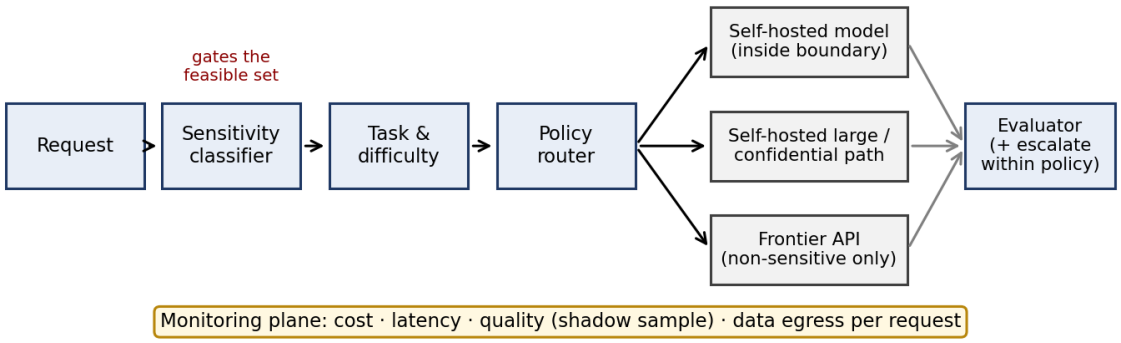


Figure 1. Security-constrained, cost-aware routing. The sensitivity classifier runs first and fixes the set of permitted destinations; cost and quality are optimized only within that set. The evaluator can escalate, but only along permitted paths.

The tier fixes the set of destinations the request may reach. Only then does the router consider cost and quality, choosing among the permitted destinations by task type and estimated difficulty. Formally, the router minimizes expected cost subject to a quality floor and a latency ceiling, with one constraint that is never traded: for restricted requests, external egress is zero. Security is a constraint on the feasible set, not a penalty term to be balanced against savings. That distinction matters, because a penalty can always be outweighed by a large enough saving and a legal obligation cannot.

Two design consequences follow. First, the classifier must be tuned for recall on the restricted tier. A false negative leaks regulated data; a false positive merely keeps a benign prompt on a local model at a small cost. Teams should therefore accept a deliberately conservative classifier and measure its recall on a labelled sample, combining pattern-based detectors for structured identifiers with model-based detection for free text [5]. Second, escalation must respect the boundary. If a local model's answer to a restricted request is judged insufficient, the request escalates to a larger self-hosted model or to a confidential-computing path in which data in use is protected by hardware-backed isolation and remote attestation, never to an external frontier model. Table 1 summarizes the resulting policy.

Table 1. Routing policy: sensitivity tier fixes the feasible set; difficulty selects within it.

Tier	Routine request	Hard request or failed evaluation
Restricted	Self-hosted model inside the boundary	Larger self-hosted model or confidential-computing path; never external
Internal	Self-hosted model	Frontier model only under an approved agreement, after de-identification
Public	Cheapest model that meets the quality bar	Frontier model

This framing aligns the router with zero-trust principles, in which no boundary crossing is implicit and every access decision is explicit and logged [6]. It also addresses LLM-specific leakage risks that perimeter controls miss. Prompts and retrieved context can be exfiltrated through indirect prompt injection [7,8], and data sent to a model can surface later in unexpected places [9]. Keeping restricted content inside the boundary bounds what any of these attacks can reach.

Meter egress the way you meter cost

Cost dashboards are standard; egress dashboards are rare. In this design the router records, for every request, which destination it reached and what content crossed the boundary. That turns a governance concern into a number: the fraction of requests, tokens or documents that left the organization, by tier and by application. For restricted traffic the number must be zero, and any non-zero value is an incident. For internal traffic it is a reported metric that security and product teams can negotiate over, just as they negotiate cost. Only the router sees every request and its destination, so egress belongs at the routing layer for the same reason cost does.

Egress metering also exposes an under-appreciated side effect of cost optimization. Every escalation of an internal or public request to a frontier model is both a cost event and an egress

event. A router tuned purely for savings tends to escalate generously when quality slips; a router that reports egress makes that trade visible.

Where the savings actually come from

Teams often justify routing projects with the price ratio between a frontier model and a small one, which can exceed an order of magnitude. The price ratio matters less than it seems. A simple model makes the point. Normalize the cost of serving one request on the frontier model to 1. Let s be the fraction of restricted traffic, h the fraction of non-restricted traffic routed straight to the frontier because it is judged hard, r the relative cost of the local model, v the relative cost of the evaluator, and e the fraction of local-first answers the evaluator rejects and escalates. Restricted requests escalate, at rate e_s, to a larger local model of relative cost r_L. Expected cost per request is then

$$C = s\,(r + v + e_s\, r_L) + (1 - s)\,[\,h + (1 - h)(r + v + e)\,] \qquad (1)$$

and the fraction of requests that leave the boundary is $(1 - s)[h + (1 - h)\,e]$. Two observations follow directly. For non-restricted traffic, a local-first attempt pays off only when $r + v + e < 1$; with a local model at 10% of frontier cost and a 5% evaluator, the cascade breaks even at an 85% escalation rate. And because e varies across tens of percentage points in practice while r varies across only a few, escalation is the lever teams most often underestimate.

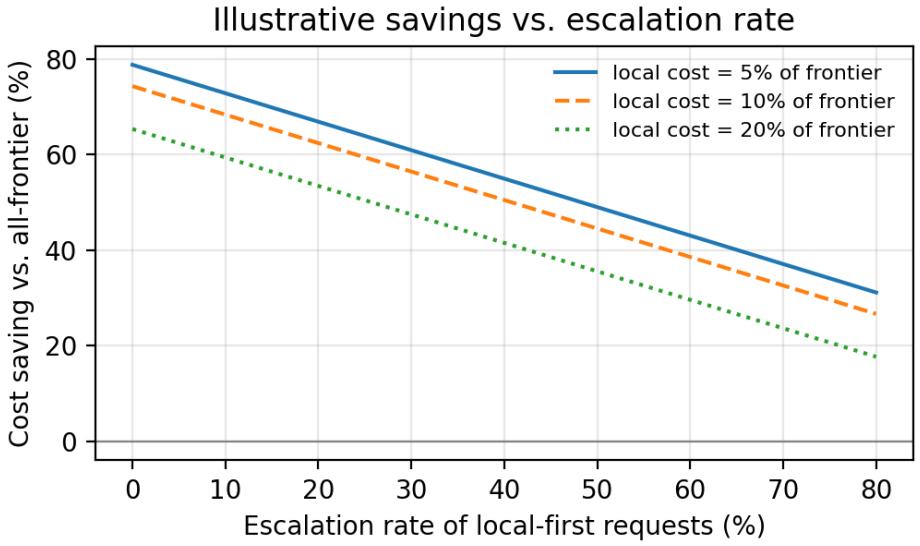


Figure 2. Illustrative savings from Equation 1 with 30% restricted traffic, 15% of the remaining traffic routed straight to the frontier, a 5% evaluator overhead and restricted escalations to a local model at 30% of frontier cost. Values are computed from the model, not measured.

Figure 2 plots Equation 1 for one illustrative mix. With the local model at 10% of frontier cost, savings fall from 74% at zero escalation to 62% at 20% and 39% at 60%. Raising the local model's relative cost from 5% to 20% lowers savings by about 13 points at 20% escalation; raising escalation from 10% to 40% lowers them by about 18 points, and over that range the share of requests leaving the boundary doubles, from 16% to 34%. The parameters are assumptions chosen for illustration; teams should substitute their own traffic mix and prices. The shape of the result does not depend on them: the cheapest lever is a better difficulty estimate, because it lowers both cost and egress at once.

Difficulty estimation is therefore where engineering effort pays back. Signals include task type, prompt features, token-level entropy and the agreement of multiple samples or multiple models [10,11]. Calibration is the property to measure. A router that trusts an overconfident local model under-escalates and lowers quality; one that distrusts a capable local model over-escalates and loses both savings and boundary discipline. Consulting a second model only when the first is uncertain is one practical way to buy calibration cheaply; in a related study on coding agents, uncertainty-gated consultation let lightweight models match a frontier model's task resolution at substantially lower cost [12].

The failure you will not see

A routing system rarely fails loudly. When a cheap model's answer is wrong but plausible and the evaluator accepts it, nothing breaks; quality simply drifts down for the traffic the router kept local. That silent regression is the main operational risk of cost-aware routing, and it is invisible in cost and latency dashboards, which improve as it worsens.

The defence is continuous shadow evaluation. Sample routed requests, replay them against a reference (the frontier model, a stronger local model or a human reviewer, depending on the task) and compare outcomes by task type and tier. Sample size follows from ordinary binomial estimation: to estimate a success rate near 90% to within ±2 percentage points at 95% confidence requires about 865 sampled requests per task type per reporting period. Restricted samples must be replayed only against references inside the boundary. Shadow evaluation also provides the labelled data needed to recalibrate the difficulty estimator, closing the loop that keeps escalation, cost and egress honest. It belongs in the system's definition of done, alongside unit tests and

dashboards, and maps naturally onto the measurement and management functions of the NIST AI Risk Management Framework [13].

Engineering the boundary in practice

Self-hosting is now practical for routine enterprise tasks. Open-weight model families are competitive on summarization, extraction, classification and much everyday code assistance, and modern serving engines with continuous batching and paged attention make them economical on enterprise hardware [14]. The engineering effort lies around the model: authentication, retrieval over internal data, connectors, quotas, monitoring and the security controls that make the self-hosted tier trustworthy in its own right.

Three implementation details deserve emphasis. Keep the classifier, router and evaluator as separate, testable components with their own metrics, so that a change in one cannot silently weaken another. Version routing policy as code and review it like any security policy, because a one-line threshold change can move both spend and egress. And treat prompt injection as a routing input: content retrieved from untrusted sources should not be able to change a request's tier or destination, a property worth testing with injected documents, following the OWASP guidance for LLM applications [8].

What to put on the dashboard, and how to roll it out

Table 2 lists the indicators we recommend reporting per application, task type and tier. The first three are familiar from any cost project. The last three are what make the router trustworthy: they show that the boundary held, that the classifier is doing its job and that quality has not drifted.

Table 2. Indicators for a security-constrained router, reported per application, task type and sensitivity tier.

Indicator	Definition	Target or use
Cost per request	Spend normalized to the all-frontier baseline	Trend down; explain regressions
Escalation rate	Share of local-first requests escalated	Primary savings driver; alert on jumps
p95 latency	Tail latency including escalations	Within the interactive budget
Egress fraction	Share of requests and tokens leaving the boundary	Zero for restricted; reported for others
Classifier recall (restricted)	Recall on a labelled audit sample	Near 100%; precision may be lower
Shadow quality delta	Success rate of routed answers minus reference	Within an agreed tolerance per task type

Rollout should follow the same progressive discipline as any change to a production path. Start in shadow mode: route every request as today, but run the classifier and router alongside and log what they would have done. This produces the first egress and escalation estimates without risk, and gives the security team a labelled sample on which to measure classifier recall. Next, enable routing for the public tier only, where a mistake costs money but cannot leak regulated data, and compare shadow quality against the baseline for at least one full reporting period. Extend to the internal tier once the data-processing agreements and de-identification steps are in place, and to the restricted tier last, after the self-hosted path has been load-tested and its own security controls reviewed. At each step, the change is reversible by switching the router back to its previous policy version.

A note on ownership. The router sits between product, platform and security teams, and it works best when each owns one part of its configuration: security owns the tier definitions and the classifier's recall target, platform owns the destinations and their capacity, and product owns the quality tolerances per task type. Making those boundaries explicit in the policy repository is as much a part of trustworthy AI as any model choice.

Three practices to adopt now

1. **Classify before you optimize.** Run sensitivity classification first, tune it for recall on restricted data, and make zero egress for restricted requests a property of the architecture rather than of a threshold.
2. **Budget escalation, not just price.** Track the escalation rate as a primary indicator. A local-first cascade saves money only while local cost, evaluation overhead and escalation rate together stay below the frontier cost; improving the difficulty estimate reduces both cost and egress.
3. **Assume silent regression and measure it.** Shadow-evaluate a fixed sample per task type and tier every reporting period (about 865 requests detects a 2-point change near 90% quality) and use the results to recalibrate routing.

Routing is usually sold as a cost project. Built with the boundary first, it becomes something more useful: the single place where an organization can see, control and prove what its AI systems cost, how good they are and where its data goes.

Acknowledgment

The author used an AI assistant (Claude, Anthropic) to help draft and edit this article and to produce the figures from the analytical model. The author is responsible for the content and has reviewed and verified it.

References

[1] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv:2305.05176, 2023.

[2] I. Ong *et al.*, "RouteLLM: Learning to route LLMs with preference data," in *Proc. ICLR*, 2025. arXiv:2406.18665.

[3] D. Ding *et al.*, "Hybrid LLM: Cost-efficient and quality-aware query routing," in *Proc. ICLR*, 2024.

[4] P. Aggarwal *et al.*, "AutoMix: Automatically mixing language models," in *Proc. NeurIPS*, 2024. arXiv:2310.12963.

[5] Microsoft, "Presidio: Data protection and de-identification SDK," open-source project. [Online]. Available: https://microsoft.github.io/presidio/

[6] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, 2020.

[7] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[8] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025. [Online]. Available: https://genai.owasp.org/

[9] N. Carlini *et al.*, "Extracting training data from large language models," in *Proc. USENIX Security Symposium*, 2021.

[10] S. Kadavath *et al.*, "Language models (mostly) know what they know," arXiv:2207.05221, 2022.

[11] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," in *Proc. ICLR*, 2023.

[12] P. S. L. Paladugu *et al.*, "Matching frontier code agents with lightweight models via multi-model consultation," OpenReview, 2026. [Online]. Available: https://openreview.net/pdf?id=mwZez5n2mG

[13] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, 2023.

[14] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM SOSP*, 2023.

About the author

Pujitha Sri Lakshmi Paladugu is an independent researcher in Lynnwood, Washington, USA, working on distributed cloud infrastructure, secure edge architecture and AI-inference infrastructure, with 10+ years of experience building and operating large-scale distributed systems. Her research covers cost-aware model routing, zero-trust edge architecture and the reliability of

agentic systems. She received an M.S. in computer science from the University of Texas at Dallas. Contact her at pujitha.rathna@gmail.com.